

OpenAI API en Python



Ruta del tutorial

3 Configurar API key

Instalación, variable de entorno y cliente.

4 Primera llamada

Generar una respuesta mínima.

7 Estructura

Roles, input y salida.

8 Prompting

Tarea + formato + restricciones.

9 JSON

Salida procesable por código.

10 Herramientas

web_search,
file_search, code
interpreter...

11 Function calling

Conectar funciones de tu backend.

```
from openai import OpenAI

client = OpenAI()

response = client.responses.create(
    model="gpt-5.5",
    input="Hola, API"
)
```

Punto 3

Configurar la API key

No expongas la clave en el frontend ni en repositorios.

- Instala el SDK oficial de Python.
- Guarda OPENAI_API_KEY como variable de entorno.
- Crea un cliente OpenAI() y úsalo desde tu backend.
- Lee secretos desde el entorno, nunca hardcodeados.

```
# Terminal
pip install openai
export OPENAI_API_KEY="sk-..."

# app.py
from openai import OpenAI

client = OpenAI()
```

Clave práctica

Tu app llama a tu backend; tu backend llama a OpenAI.

Producción

Usa variables de entorno, secret manager y rotación de claves.

Punto 4

Primera llamada al modelo

La mínima integración: model + input → output_text.

- `responses.create()` crea una respuesta del modelo.
- `model` define qué modelo usar.
- `input` puede ser texto simple o una lista de mensajes.
- `output_text` contiene el texto final más cómodo de leer.

```
from openai import OpenAI

client = OpenAI()

response = client.responses.create(
    model="gpt-5.5",
    input="Explícame qué es una API en una frase."
)

print(response.output_text)
```

Patrón base

Cliente → petición → respuesta → render en tu app

Punto 7

Estructura de una petición

Usa mensajes para separar instrucciones y petición del usuario.

```
from openai import OpenAI

client = OpenAI()

response = client.responses.create(
    model="gpt-5.5",
    input=[
        {
            "role": "developer",
            "content": "Responde en español claro."
        },
        {
            "role": "user",
            "content": "¿Qué es machine learning?"
        }
    ]
)

print(response.output_text)
```

developer

Reglas de comportamiento de la app: idioma, tono, formato, límites.

user

La petición concreta del usuario final.

assistant

Puede aparecer en historial para mantener contexto conversacional.

output_text

Forma simple de extraer la respuesta textual final.

Consejo: guarda tus instrucciones estables en developer y deja el input del usuario separado.

Prompting básico

Haz explícitos tarea, formato, tono y restricciones.

Plantilla mental

Tarea → Formato → Tono → Restricciones → Datos

- Define exactamente qué debe hacer.
- Pide un formato verificable: bullets, tabla o JSON.
- Prohíbe inventar datos si hay contexto fuente.
- Da ejemplos cuando el formato sea importante.

```
from openai import OpenAI

client = OpenAI()

texto = """...pega aquí el texto largo..."""

response = client.responses.create(
    model="gpt-5.5",
    input=f"""
    Resume el texto en 5 puntos.
    Usa lenguaje sencillo.
    No añadas información externa.

    Texto:
    {texto}
    """
)

print(response.output_text)
```

JSON estructurado

Convierte lenguaje natural en datos que tu app puede validar.

- Pide claves concretas y tipos esperados.
- Valida con `json.loads()` antes de usarlo.
- Maneja errores si el texto de entrada está incompleto.
- Para producción, usa esquemas estructurados cuando aplique.

El beneficio no es “texto bonito”: es automatización.

```
from openai import OpenAI
import json

client = OpenAI()

texto = """
Ana García, ana@example.com, trabaja en Acme
y necesita soporte urgente.
"""

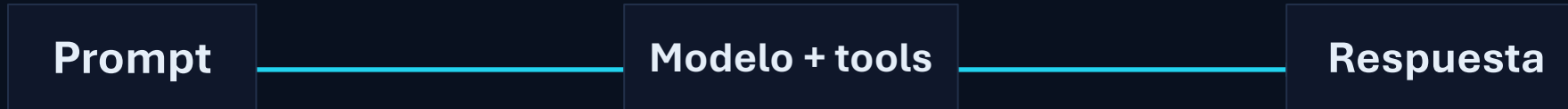
response = client.responses.create(
    model="gpt-5.5",
    input=f"""
Extrae datos y devuelve SOLO JSON válido:
{{"nombre": str, "email": str,
  "empresa": str, "prioridad": str}}

Texto: {texto}
"""
)

datos = json.loads(response.output_text)
print(datos["email"])
```

Herramientas: más allá del texto

Permiten buscar, leer archivos o ejecutar capacidades externas.



- web_search: información actualizada de Internet.
- file_search: tus documentos y vector stores.
- code_interpreter: Python para análisis y archivos.
- image_generation, computer, shell y MCP según el caso.

```
from openai import OpenAI

client = OpenAI()

response = client.responses.create(
    model="gpt-5.5",
    tools=[{"type": "web_search"}],
    input="Busca novedades recientes de Python."
)

print(response.output_text)
```

Usa tools cuando el modelo necesite datos externos, documentos o capacidades que no están en el prompt.

Web search: ejemplo básico

El modelo consulta Internet cuando necesita información actualizada.

- Añade la herramienta con `tools=[{"type": "web_search"}]`.
- Pide explícitamente que use fuentes y resume hallazgos.
- Ideal para noticias, documentación cambiante o datos recientes.
- Lee `response.output_text` como en cualquier respuesta.

Patrón: mismo `create()`, pero con una tool integrada.

```
from openai import OpenAI

client = OpenAI()

response = client.responses.create(
    model="gpt-5.5",
    tools=[{"type": "web_search"}],
    input="""
Busca información actualizada sobre Python.
Resume 3 novedades con fuentes.
"""
)

print(response.output_text)
```

Web concreta: filtrar por dominio

Limita la búsqueda a una fuente específica usando `allowed_domains`.

- Usa `filters.allowed_domains` para acotar la web consultada.
- Escribe dominios sin `https://`; por ejemplo `docs.python.org`.
- Útil para documentación oficial, bases de conocimiento públicas o portales confiables.
- Pide que no invente fuera de las fuentes encontradas.

Control = menos ruido, más confianza.

```
from openai import OpenAI

client = OpenAI()

response = client.responses.create(
    model="gpt-5.5",
    tools=[{
        "type": "web_search",
        "search_context_size": "medium",
        "filters": {
            "allowed_domains": ["docs.python.org"]
        }
    }],
    input="""
Consulta la documentación oficial de Python.
Explica pathlib.Path.read_text con ejemplo.
Incluye fuentes.
"""
)

print(response.output_text)
```

Herramientas principales

Cada tool amplía una capacidad distinta del modelo.

web_search	Buscar información actual en Internet.
file_search	Recuperar datos de tus documentos/vector stores.
function	Llamar funciones de tu backend con JSON schema.
code_interpreter	Ejecutar Python para análisis, cálculos o archivos.
image_generation	Generar o editar imágenes desde la API.
computer / shell	Interactuar con UI o ejecutar comandos en entornos controlados.
mcp	Conectar herramientas externas mediante servidores MCP.

```
# La estructura siempre se parece:

response = client.responses.create(
    model="gpt-5.5",
    input="Tu tarea...",
    tools=[
        {"type": "web_search"},
        {"type": "file_search",
         "vector_store_ids": ["vs..."]}
    ]
)
```

Regla de decisión

Si el dato está fuera del prompt, dale una tool.
Si la acción es de negocio, usa function calling.

Ejemplo mínimo: file search

El modelo busca en tus archivos subidos a un vector store.

- Crea un vector store para tu base de conocimiento.
- Añade archivos al vector store y espera a que estén listos.
- Pasa vector_store_ids en la herramienta file_search.
- Puedes incluir resultados con include=["file_search_call.results"].

Uso típico: documentos internos, contratos, FAQs, manuales o papers.

```
from openai import OpenAI

client = OpenAI()

vector_store = client.vector_stores.create(
    name="knowledge_base"
)

client.vector_stores.files.create(
    vector_store_id=vector_store.id,
    file_id="file_..."
)

response = client.responses.create(
    model="gpt-5.5",
    input="Resume las obligaciones del contrato.",
    tools=[{
        "type": "file_search",
        "vector_store_ids": [vector_store.id]
    }]
)

print(response.output_text)
```

Function calling

El modelo decide cuándo pedir a tu backend que ejecute una función.

- Defines funciones disponibles con nombre, descripción y parámetros.
- El modelo devuelve una tool call con argumentos.
- Tu código ejecuta la función real.
- Devuelves el resultado al modelo para redactar la respuesta final.

Importante: el modelo no accede solo a tu base de datos; tu backend controla qué se ejecuta.

```
from openai import OpenAI

client = OpenAI()

tools = [{
    "type": "function",
    "name": "get_order_status",
    "description": "Obtiene el estado de un pedido",
    "parameters": {
        "type": "object",
        "properties": {
            "order_id": {"type": "string"}
        },
        "required": ["order_id"],
        "additionalProperties": False
    }
}]

response = client.responses.create(
    model="gpt-5.5",
    input="¿Estado del pedido A123?",
    tools=tools
)

print(response.output)
```

Ejemplo mínimo: function calling

Tu backend ejecuta la función; el modelo solo solicita la llamada.

- Define la función con name, description y parameters.
- El modelo devuelve un item de tipo function_call.
- Ejecuta tu código con los argumentos recibidos.
- Devuelve el resultado como function_call_output.

En producción, haz una segunda llamada con el resultado para que el modelo redacte la respuesta final.

```
from openai import OpenAI
import json

client = OpenAI()

def get_order_status(order_id):
    return {"status": "en reparto"}

tools = [{
    "type": "function",
    "name": "get_order_status",
    "description": "Consulta un pedido.",
    "parameters": {
        "type": "object",
        "properties": {"order_id": {"type": "string"}},
        "required": ["order_id"],
        "additionalProperties": False
    }
}]

response = client.responses.create(
    model="gpt-5.5",
    input="¿Dónde está el pedido A123?",
    tools=tools
)

for item in response.output:
    if item.type == "function_call":
        args = json.loads(item.arguments)
        print(get_order_status(**args))
```